CURSO BIS08 · ESTRUCTURAS DE DATOS · QUIZ 2

Parque de Diversiones

Quiz 2 — Pilas, Colas y Recursividad. Elijan equipo, programen el parque, súbanlo a GitHub.

La idea

Ustedes son el equipo de programación de un parque de diversiones. Hoy el parque abre sus puertas, durante el día va a recibir visitantes, va a tener algunos reclamos que atender, y en la noche va a cerrar con un gran espectáculo de fuegos artificiales. Van a programar una única aplicación que simule ese día completo, y que por dentro use exactamente las tres herramientas que vimos en clase: una cola, una pila y una función recursiva.

No es un programa separado por cada tema: es UN solo programa — el parque — donde cada parte cumple un trabajo distinto, igual que en cualquier sistema real.

¿Qué tienen que hacer?

1. Formen equipos de 2 o 3 personas.
2. Programen, en Python, un simulador de "Un día en el parque" con un menú simple que incluya las tres partes descritas más abajo: la fila de la montaña rusa (cola), el registro de reclamos con función "deshacer" (pila), y el cierre con fuegos artificiales (recursividad).
3. La cola y la pila las tienen que implementar ustedes mismos (con nodos enlazados o con una lista, como prefieran) — no alcanza con usar únicamente .append()/.pop() de Python sin explicar o estructurar esa lógica como una cola o una pila propia.
4. La recursividad del espectáculo final TIENE que ser una función que se llama a sí misma — no un for ni un while disfrazado.
5. Suban su código a un repositorio de GitHub, con un archivo README.md corto que explique: qué hace el programa, cómo correrlo, y qué instalar (si hace falta).
6. Al final de la clase, cada equipo comparte pantalla entre 2 y 3 minutos: muestra el parque funcionando (gente entrando a la fila, un reclamo que se deshace, y el cierre con fuegos artificiales) y señala en el código dónde está la cola, dónde está la pila y dónde está la recursión.

El parque que van a construir

Las tres partes conectan con la misma historia: gente que entra, cosas que hay que poder deshacer, y un final que se construye solo, en base a sí mismo.

Parte 1 — La fila de la montaña rusa (COLA)

Los visitantes van llegando y se forman en una fila para subir a la atracción principal. Suben en el mismo orden en que llegaron: el primero que llega es el primero que sube (FIFO).

Debe poder: agregar un visitante al final de la fila, subir al carrito al visitante que lleva más tiempo esperando (el primero de la fila), y mostrar en pantalla quién está esperando en este momento.

Pista: Es la misma idea de la cola de atención que vimos en clase, solo que ahora las "personas en espera" son visitantes y el "servidor" es el carrito de la montaña rusa.

Parte 2 — El libro de reclamos con "deshacer" (PILA)

Durante el día pasan cosas: alguien compra un boleto, alguien hace un reclamo, alguien pide un reembolso. El encargado del parque quiere poder "deshacer" lo último que se registró, como un Ctrl+Z, sin tener que deshacer todo lo anterior.

Debe poder: registrar una acción nueva (por ejemplo: "Boleto vendido a Juan"), deshacer la acción más reciente (y mostrar cuál se deshizo), y mostrar el historial completo de acciones que siguen vigentes.

Pista: Lo último que entró es lo primero que se puede deshacer (LIFO) — igual que la pila de platos o la pila de "deshacer" de un editor de texto que vimos en clase.

Parte 3 — El cierre: fuegos artificiales (RECURSIVIDAD)

Al cerrar el parque se lanza un cohete que explota en varias partículas más chicas, y cada una de esas también explota en partículas más chicas todavía, hasta que son demasiado pequeñas y se apagan — igual que el árbol fractal o el Sierpinski que programaron en la Clase 8.

Debe poder: dibujar (con turtle, o con texto/colores en consola si prefieren) la explosión inicial, y llamar recursivamente a la misma función sobre cada partícula hija, con un tamaño más chico cada vez, hasta llegar al caso base.

Pista: El caso base es "la partícula ya es demasiado chica": ahí la función no crea más hijas y simplemente se apaga.

¿Qué se va a evaluar?

- Que la cola de la fila funcione de verdad en orden FIFO (quien llega primero, sube primero).
- Que la pila de reclamos funcione de verdad en orden LIFO (se deshace siempre lo más reciente).
- Que los fuegos artificiales usen una función recursiva real, con un caso base claro.
- Que el programa corra sin errores y que el README explique con claridad cómo probarlo.
- Qué tan creativo o entretenido queda el resultado (nombres divertidos, mensajes con onda, colores, dibujos) — esto no reemplaza los puntos anteriores, los complementa.

Ideas para ir más allá (opcional)

- Agregar una segunda fila (por ejemplo, para una atracción "VIP") y comparar tiempos de espera.
- Que el tamaño o color de los fuegos artificiales dependa de cuántos visitantes subieron en el día.
- Guardar el historial de reclamos en un archivo de texto al cerrar el parque.

Cómo subir su proyecto a GitHub (resumen rápido)

- Creen un repositorio nuevo en github.com (botón "New repository"), público, con un nombre corto relacionado a su proyecto.
- En su computadora, dentro de la carpeta del proyecto, ejecuten: git init, luego git add . y luego git commit -m "Primera version".
- Conecten su carpeta con el repositorio que crearon (GitHub les muestra el comando exacto apenas crean el repositorio; normalmente es git remote add origin seguido del link de su repositorio).
- Suban el código con: git push -u origin main.
- Confirmen que el código se ve correctamente en la página de su repositorio antes de compartir el link con la clase.